Generación de imágenes sintéticas de fondo de ojo mediante WGAN-GP para mejorar la disponibilidad de datos en el diagnóstico de retinopatía diabética

Mauro Carlevaro

Maestría en Ciencias Cognitivas

Facultad de Ciencias, Ingeniería y Psicología

UdelaR, Uruguay

Email: mauroctecno@gmail.com

***Resumen*— La escasez de imágenes médicas etiquetadas constituye una de las principales limitaciones para el desarrollo de sistemas de diagnóstico asistido por computadora basados en aprendizaje profundo. En este trabajo se aborda dicha limitación mediante modelos generativos adversariales (GANs) para sintetizar imágenes de fondo de ojo realistas. Se implementan y comparan tres modelos: DCGAN (baseline), WGAN con weight clipping (WGAN-clip) y WGAN con penalización de gradiente (WGAN-GP), entrenados sobre el dataset Messidor-2 (1.744 imágenes). Se realizó una exploración sistemática de hiperparámetros para WGAN-GP variando lambda_gp y n_critic. La evaluación se realiza mediante Fréchet Inception Distance (FID), varianza promedio de píxeles y distancia promedio entre imágenes como métricas de diversidad generativa.**

Los resultados evidencian que WGAN-clip colapsa, mientras que WGAN-GP con la configuración óptima identificada (lambda_gp=10, n_critic=3, InstanceNorm) alcanza FID=87.06, aproximándose al baseline DCGAN (FID=47.40), evidenciando el impacto de la función objetivo basada en distancia de Wasserstein y la importancia de la configuración experimental en escenarios médicos con datos limitados.

Se busca ampliar la cantidad de imágenes de forma de contribuir a equilibrar el dataset clínico, con el potencial de mejorar la capacidad de los modelos para detectar lesiones tempranas, aumentar la sensibilidad diagnóstica y favorecer la capacidad del modelo de mantener su desempeño en imágenes provenientes de diferentes fuentes, lo que aportaría valor en entornos con escasez de especialistas. Estos hallazgos evidencian el potencial de los modelos generativos para ampliar datasets y apoyar el entrenamiento de sistemas de diagnóstico asistidos por IA.

Index Terms*— *Modelos Generativos Adversariales; Estabilidad en GAN; WGAN-GP; Síntesis de Imágenes Médicas; Retinopatía Diabética; Aprendizaje Profundo.

***Abstract*— The scarcity of labeled medical images is one of the main limitations for the development of deep learning based computer-aided diagnosis systems. This work addresses this limitation using Generative Adversarial Networks (GANs) to synthesize realistic retinal fundus images. Three models are implemented and compared: DCGAN (baseline), WGAN with weight clipping (WGAN-clip), and WGAN with gradient penalty (WGAN-GP), trained on the Messidor-2 dataset (1,744 images). A systematic hyperparameter exploration for WGAN-GP was conducted by varying lambda_gp and n_critic. Evaluation is performed using Fréchet Inception Distance (FID), average pixel variance, and mean inter-image distance as generative diversity metrics.**

Results show that WGAN-clip suffers from mode collapse, whereas WGAN-GP with the identified optimal configuration (lambda_gp = 10, n_critic = 3, InstanceNorm) achieves FID = 87.06, approaching the DCGAN baseline (FID = 47.40), highlighting the impact of the Wasserstein-based objective function and the importance of experimental configuration in medical scenarios with limited data.

The aim is to expand the number of available images to contribute to balancing the clinical dataset, with the potential to improve the ability of models to detect early lesions, increase diagnostic sensitivity, and promote the model's capacity to maintain performance across images from different acquisition sources, which would be valuable in settings with limited specialist availability. These findings demonstrate the potential of generative models to expand datasets and support the training of AI assisted diagnostic systems.

Index Terms*— *Generative Adversarial Networks; GAN Stability; WGAN-GP; Medical Image Synthesis; Diabetic Retinopathy; Deep Learning.

I. Introducción

La retinopatía diabética es una de las principales causas de ceguera prevenible a nivel mundial. La detección temprana de esta patología es clave para evitar la pérdida irreversible de la visión. En los últimos años, los sistemas de diagnóstico asistido por computadora basados en aprendizaje profundo muestran un gran potencial para apoyar la detección automática de enfermedades. No obstante, estos sistemas requieren grandes volúmenes de imágenes etiquetadas por expertos para alcanzar un desempeño confiable.

Los modelos generativos, y en particular las Redes Generativas Antagónicas (GANs), ofrecen una alternativa para mitigar este problema mediante la generación de imágenes sintéticas que aprendan la distribución de los datos reales. En el contexto de imagenes médicas, la generación de datos sintéticos puede contribuir a aumentar la diversidad del conjunto de entrenamiento sin comprometer la privacidad de los pacientes.

En este trabajo se propone el uso de WGAN-GP (Wasserstein Generative Adversarial Network con penalización de gradiente) para la generación de imágenes sintéticas de fondo de ojo y se compara su desempeño con DCGAN y con WGAN estándar con weight clipping, representando las principales estrategias de entrenamiento adversarial basadas en la formulación GAN clásica y en la distancia de Wasserstein.

II. Descripción del conjunto de datos

El conjunto de datos utilizado es Messidor-2, un dataset clínicamente validado de imágenes de fondo de ojo ampliamente empleado como punto de referencia estándar en investigación médica. Publicado en 2014, contiene 1.744 imágenes RGB, etiquetadas según la severidad de la retinopatía diabética en grados de 0 a 4.

Messidor-2 continúa siendo relevante debido a la alta calidad de sus imágenes, la consistencia del protocolo de adquisición y la confiabilidad de las anotaciones clínicas realizadas por expertos. Estas características lo convierten en un conjunto de datos adecuado para evaluar modelos generativos en un entorno médico realista.

Con el objetivo de caracterizar el conjunto de datos, se analizó la distribución de las clases de severidad presentes en Messidor-2. La siguiente figura muestra la distribución de imágenes por grado de retinopatía diabética. Se observa una distribución desbalanceada, lo cual refleja un escenario clínico realista y constituye un desafío adicional para modelos generativos y discriminativos.

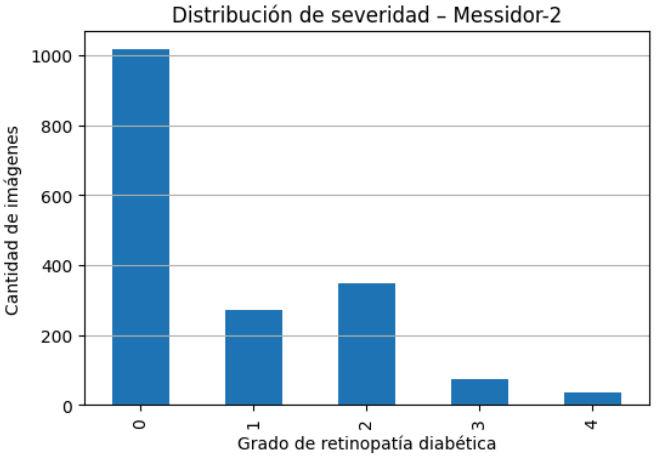


Figura 1. Distribución de severidad Messidor-2

En la figura 1 observamos una distribución no balanceada entre las clases, predominando el grado 0 (sin retinopatía). Esta distribución evidencia la limitada disponibilidad de imágenes en determinados grados de severidad, lo que refuerza la relevancia de explorar técnicas de generación sintética para aumentar la diversidad del conjunto de entrenamiento.

Posteriormente con el objetivo de caracterizar estadísticamente las propiedades fotométricas del conjunto de datos, se analizó la distribución del brillo promedio y la varianza de intensidad en las imágenes (utilizada como aproximación del contraste). La Figura 2 muestra la distribución del brillo promedio, evidenciando una variabilidad significativa entre imágenes.

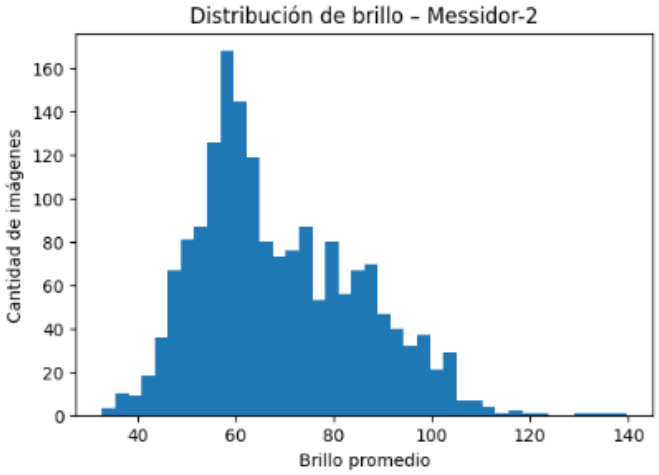


Figura 2. Distribución de brillo Messidor-2

La Figura 3 presenta la distribución del contraste, donde se observa dispersión asociada a diferencias en condiciones de adquisición e iluminación.

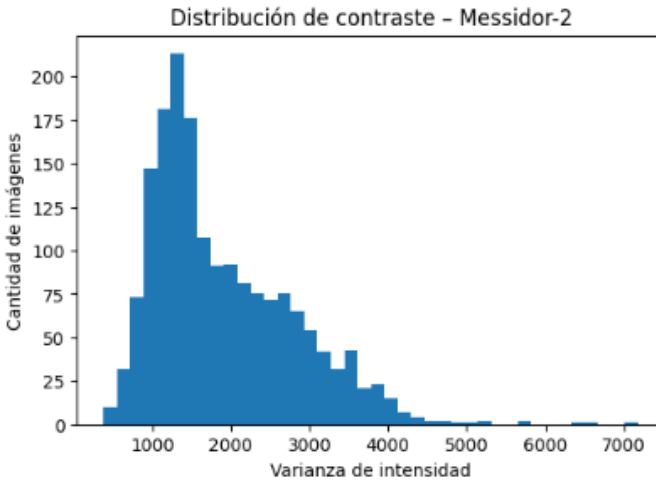


Figura 3. Distribución de contraste Messidor-2

Esta heterogeneidad constituye un desafío adicional para modelos generativos, ya que deben aprender no solo la estructura anatómica de la retina sino también la variabilidad en condiciones de captura.

Para finalizar esta sección tenemos en la Figura 4 ejemplos representativos de imágenes con bajo brillo, alto brillo y bajo contraste, evidenciando la variabilidad en condiciones de adquisición. Esta heterogeneidad constituye un desafío adicional para el modelado generativo.

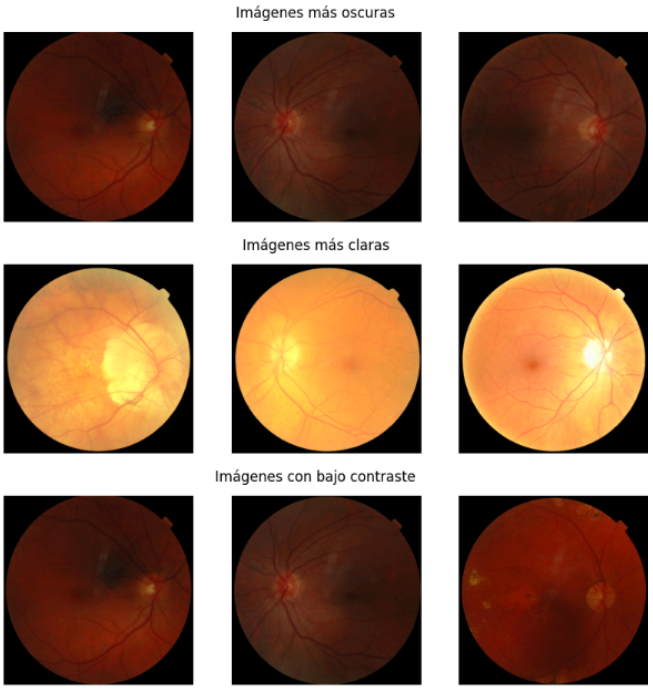


Figura 4. Variabilidad del conjunto de imágenes

III. Metodología

La metodología propuesta comprende las siguientes etapas: (1) análisis exploratorio del dataset evaluando tamaño, balance de clases, resolución y propiedades fotométricas;

(2) preprocesamiento con redimensionado a 128×128 píxeles y normalización a [−1,1];

(3) implementación de tres modelos generativos: DCGAN como baseline, WGAN-clip como comparación intermedia en dos variantes, y WGAN-GP como modelo propuesto (por su estabilidad en el entrenamiento adversarial) con cuatro configuraciones de hiperparámetros;

(4) exploración sistemática de hiperparámetros WGAN-GP variando lambda_gp = 5 y 10, y n_critic = 3 y 5 (valores estándar de estabilidad en el entrenamiento adversarial) ;

(5) entrenamiento bajo condiciones controladas con generador compartido, misma resolución, batch size y número de épocas;

(6) evaluación cuantitativa mediante FID y métricas de diversidad, y cualitativa mediante comparación visual con vector latente fijo;

(7) generación de un dataset sintético de 2.000 imágenes con la mejor configuración identificada.

A. Preprocesamiento

Las imágenes son redimensionadas a 128×128 píxeles y normalizadas al rango [−1, 1], lo cual resulta adecuado porque en DCGAN y WGAN-GP el generador utiliza la función de activación tanh (tangente hiperbólica) en su última capa (salida del generador) y produce las salidas en ese mismo rango. Alinear el rango de los datos reales con el de los datos generados facilita el aprendizaje del modelo y evita inconsistencias durante el entrenamiento.

Esta normalización estándar favorece la estabilidad numérica reduciendo la saturación de activaciones y favoreciendo un flujo de gradientes más suave lo que mejora la convergencia del entrenamiento.

Finalmente, esta elección corresponde a una práctica estándar en GANs convolucionales, establecida desde el trabajo original de DCGAN.

B. Modelo base: DCGAN

Como modelo base se implementa una DCGAN estándar, donde tanto el generador como el discriminador están compuestos por capas convolucionales y convolucionales transpuestas, con normalización por lotes y funciones de activación ReLU y LeakyReLU.

El discriminador incluye Sigmoid en la salida y optimiza una pérdida BCE (Binary Cross Entropy). Se aplica label smoothing para mejorar la estabilidad del entrenamiento.

La BCE presenta una limitación cuando el discriminador se vuelve muy preciso, los gradientes que llegan al generador tienden a cero, dificultando el aprendizaje. Esta limitación motiva el uso de la distancia de Wasserstein en WGAN-GP, que proporciona gradientes más informativos incluso cuando las distribuciones real y generada están muy separadas.

El generador y el crítico siguen una arquitectura convolucional profunda con NGF=NDF=64 filtros base, valor estándar usado en todas las arquitecturas del paper original de DCGAN. El generador reduce progresivamente los filtros a medida que aumenta la resolución espacial (512→256→128→64→32→3 canales), partiendo de una representación abstracta a partir del vector latente y terminando en una imagen RGB de 128×128 píxeles. El crítico realiza el proceso inverso, aumentando filtros a medida que reduce la resolución espacial (3→64→128→256→512→1024→1 escalar), extrayendo características progresivamente más abstractas hasta producir un único valor de salida.

C. WGAN con weight clipping

Se implementa una WGAN con weight clipping como punto de comparación intermedio entre DCGAN y WGAN-GP. Esta variante impone la condición de 1-Lipschitz mediante recorte de pesos del crítico (weight clipping) en lugar de penalización de gradiente. El optimizador utilizado es RMSprop según la recomendación original del paper (Arjovsky). Se evaluaron dos configuraciones: la estándar (lr = 5e-5, clip_value = 0.01) correspondiente al paper original, y una variante con hiperparámetros ajustados (lr = 1e-4, clip_value = 0.05) para analizar el efecto de un recorte de pesos (clip_value) menos restrictivo y una mayor tasa de aprendizaje (lr) en la capacidad y estabilidad del crítico.

D. Modelo propuesto: WGAN-GP

El modelo propuesto se basa en una WGAN-GP, una variante de las GAN tradicionales diseñada para mejorar la estabilidad del entrenamiento y proporcionar gradientes más informativos. Esta reemplaza el weight clipping por una penalización de gradiente para imponer la condición de Lipschitz. La función de pérdida completa del crítico es:

$$L(P_r, P_g) = \mathbb{E}_{x \in P_r}[f_w(x)] - \mathbb{E}_{z \in P_y(z)}[f_w(G_\theta(z))]$$

A diferencia de las GAN clásicas, que optimizan una divergencia basada en entropía cruzada, WGAN optimiza la distancia de Wasserstein-1 (Earth Mover's Distance) entre la distribución real *Pr* y la distribución generada *Pg*. En este marco, el discriminador es reemplazado por un crítico *fw*, que produce valores reales no acotados. Donde G_θ(z) representa la salida del generador a partir de un vector latente z muestreado de la distribución *Pz*.

Para garantizar que el crítico cumpla la condición de 1-Lipschitz, requerida por la formulación teórica de la distancia de Wasserstein, WGAN-GP incorpora un término de penalización de gradiente. La función de pérdida completa utilizada es:

$$L(P_r, P_g) = \mathbb{E}_{x \sim P_r}[f_w(x)] - \mathbb{E}_{x \sim P_z}[f_w(G_\theta(z))] + \lambda\, \mathbb{E}_{\hat{x} \sim P_{\hat{x}}}\left[(\|\nabla_{\hat{x}} f_w(\hat{x})\|_2 - 1)^2\right]$$

donde:

- λ es el coeficiente de penalización,
- *x*^ son muestras interpoladas entre datos reales y generados,
- Px^ es la distribución inducida por dichas interpolaciones,
- el término de penalización fuerza la norma del gradiente del crítico respecto a *x*^ a aproximarse a 1.

Esta formulación reemplaza el weight clipping utilizado en la WGAN original, mejorando significativamente la estabilidad del entrenamiento y reduciendo oscilaciones y colapso de modos.

Durante el entrenamiento se realizan múltiples actualizaciones del crítico por cada actualización del generador, lo que permite una mejor aproximación de la distancia de Wasserstein antes de modificar los parámetros del generador.

Las arquitecturas del generador y del crítico siguen un diseño convolucional profundo de tipo DCGAN, basado en capas convolucionales y convolucionales transpuestas con normalización y activaciones ReLU/LeakyReLU. Este diseño estructural se adapta al marco de WGAN-GP mediante la eliminación de la función sigmoide en la salida del crítico, y la incorporación del término de penalización de gradiente en el entrenamiento.

Para la optimización se utiliza Adam con beta1=0 y beta2=0.9 para ambas redes, el valor beta1=0 elimina el momento de primer orden, evitando que gradientes históricos sesguen las actualizaciones durante el entrenamiento adversarial, lo que mejora la estabilidad cuando el crítico y el generador compiten activamente. Esta configuración difiere del Adam estándar (beta1=0.5) utilizado en DCGAN. Para WGAN-clip se utiliza RMSprop según Arjovsky, que no acumula momento y resulta más adecuado para el esquema de weight clipping.

E. Corrección de normalización en el crítico

Una corrección crítica respecto a la versión inicial fue la mencionada sustitución de BatchNorm2d por InstanceNorm2d (affine=True) en el crítico de WGAN-GP. BatchNorm normaliza usando estadísticas globales del batch, introduciendo dependencias entre muestras que corrompen el cálculo del gradient penalty e impiden una estimación confiable de la condición de Lipschitz. Esta fue la causa directa de los picos anómalos observados en la pérdida del crítico y del FID elevado (138.14) en la versión inicial del trabajo.

InstanceNorm normaliza cada imagen de forma independiente sin crear dependencias entre muestras, preservando la independencia requerida para un gradient penalty estable. Se creó una clase Critic_WGAN independiente con InstanceNorm2d, separada del discriminador DCGAN que mantiene BatchNorm2d. El generador es compartido entre los tres modelos para que la comparación se centre exclusivamente en la función de pérdida y el esquema de entrenamiento.

F. Generación del dataset sintético

Con la mejor configuración WGAN-GP identificada experimentalmente (config C: lambda_gp=10, n_critic=3), se generó un dataset sintético de 2.000 imágenes de fondo de ojo en formato PNG a resolución 128×128 píxeles. Este amplía el dataset real de Messidor-2. Las imágenes sintéticas se generan a partir de vectores latentes aleatorios z muestreados de una distribución normal estándar. Dado que el generador produce salidas en el rango $[-1,1]$, estas se reescalan al rango $[0,1]$ antes de su almacenamiento como imágenes.

IV. Configuración experimental

A. Partición del dataset y parámetros

El conjunto de datos se divide en subconjuntos de entrenamiento y validación utilizando un esquema reproducible (80% entrenamiento y 20% validación), manteniendo la proporción de cada clase. La partición de validación se utiliza únicamente para el monitoreo del entrenamiento y cálculo de métricas, no para ajuste de parámetros.

Parámetro	DCGAN
Optimizador	Adam(beta1=0.5, beta2=0.999)
Learning rate	2e-4
Batch size	8
Épocas	50
Norm. crítico/discrim.	BatchNorm2d
Clip/Penalización	—
n_critic	—

Tabla 1. Configuración de hiperparámetros modelo DCGAN

Parámetro	WGAN-clip	WGAN-GP
Optimizador	RMSprop	Adam(beta1=0, beta2=0.9)
Learning rate	5e-5 / 1e-4	1e-4
Batch size	8	8
Épocas	50	50
Norm. crítico/discrim	InstanceNorm2d	InstanceNorm2d
Clip/Penalización	clip_value=0.01 /0.05	lambda_gp (variable)
n_critic	5	variable

Tabla 2. Configuración de hiperparámetros modelos WGAN

B. Exploración de hiperparámetros WGAN-GP

Dado que WGAN-GP es sensible a la configuración de lambda_gp (coeficiente de penalización) y n_critic (frecuencia de actualización del crítico), se realizó una exploración con 4 configuraciones:

Config	lambda	critic	lr	Interpretación
B	10	5	1e-4	estable, crítico dominante
C	10	3	1e-4	estable + balanceado
D	5	5	1e-4	crítico flexible y dominante
E	5	3	5e-5	flexible + conservador

Tabla 3. Configuración de hiperparámetros modelo WGAN-GP

La variación de lambda_gp permitió analizar el efecto de la intensidad de la penalización de gradiente sobre la estabilidad del crítico, mientras que n_critic controló el equilibrio de entrenamiento entre crítico y generador. Asimismo, la tasa de aprendizaje moduló la estabilidad de convergencia. Los resultados muestran que la configuración lambda_gp=10 y n_critic=3 (Config C) logra el mejor compromiso entre estabilidad Lipschitz y balance adversarial, produciendo el menor FID. En donde:

- lambda_gp = coeficiente de penalización de Lipschitz
- n_critic = balance G/D
- lr = velocidad de convergencia, tasa de aprendizaje

La configuración C funciona mejor porque equilibra los tres: lambda_gp, n_critic y lr.

C. Presupuesto computacional y épocas de entrenamiento

Por limitaciones computacionales se evaluó el impacto de reducir el presupuesto de entrenamiento de 100 a 50 épocas. El análisis comparativo mostró que DCGAN y WGAN-clip convergen antes de la época 10, por lo que la reducción no afecta sus resultados. Para WGAN-GP la diferencia en FID fue inferior a 7 puntos (84.98 vs 90.91 para la config B), con métricas de diversidad prácticamente idénticas, evidenciando convergencia antes de la época 50. En consecuencia, se estableció un presupuesto de 50 épocas para todos los modelos.

V. Evaluación experimental

A. Estabilidad del entrenamiento

Respecto a la versión inicial, se implementaron dos mejoras en la visualización de pérdidas. Primero, se adoptó una función de visualización con dos ejes Y independientes (twinx en matplotlib), dado que la pérdida del crítico en WGAN-GP no está acotada (sin función Sigmoid) y puede tomar valores muy superiores a los del generador, haciendo ilegible una visualización con eje único. Segundo, el registro de pérdidas pasó de realizarse por batch a calcularse como promedio por época, eliminando la variabilidad de alta frecuencia que generaba picos en las curvas de la versión inicial. Inicialmente utilizando el BatchNorm incorrecto generaba inestabilidad en el entrenamiento, mientras que el logging por batch introducía ruido visual que amplificaba artificialmente dicha inestabilidad.

Para permitir una comparación visual consistente entre épocas y entre modelos, se utiliza un vector latente fijo

(fixed_noise) durante todo el entrenamiento, de modo que cada grilla de imágenes guardada corresponde exactamente a las mismas entradas latentes. Esto permite observar claramente la evolución del generador época a época y comparar directamente las salidas de distintos modelos bajo las mismas condiciones de entrada.

DCGAN exhibe oscilaciones características del entrenamiento adversarial clásico con convergencia antes de la época 10.

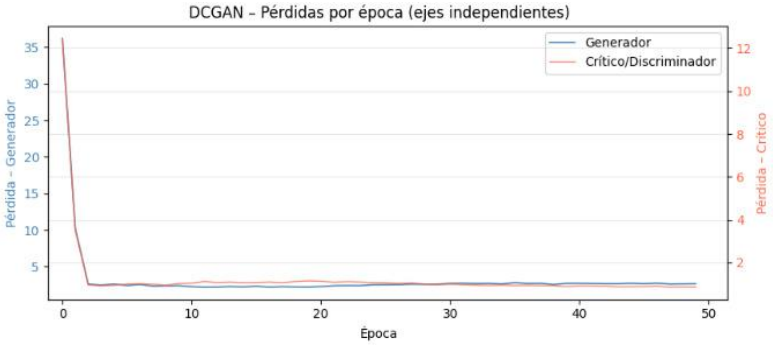


Figura 5. DCGAN baseline para comparar con WGAN-GP y WGAN-clip.

WGAN-clip colapsa desde la época 5, con pérdidas congeladas en valores constantes, comportamiento consistente con la limitación conocida del weight clipping de imponer la condición de 1-Lipschitz mediante un recorte rígido de los pesos del crítico.

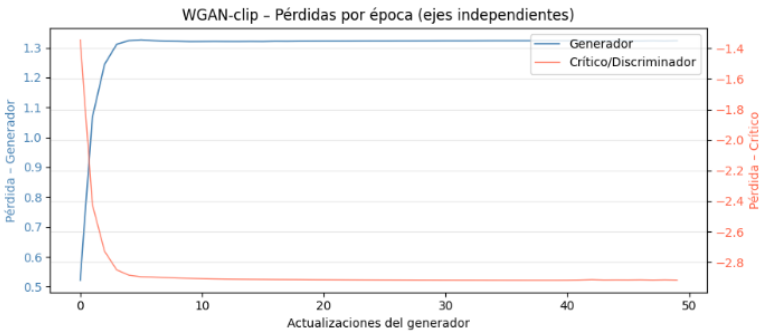


Figura 6. WGAN con weight clipping con lr=5e-5 y clip_value=0.01

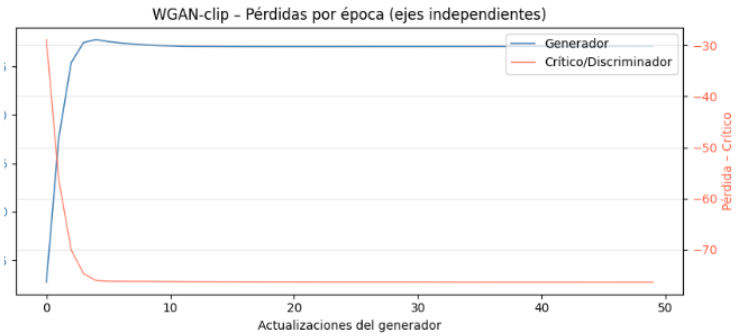


Figura 7. WGAN con weight clipping con lr=1e-4 y clip_value=0.05

WGAN-GP con InstanceNorm muestra una reducción progresiva de la distancia de Wasserstein estimada (LossC), sin los picos anómalos de la versión con BatchNorm.

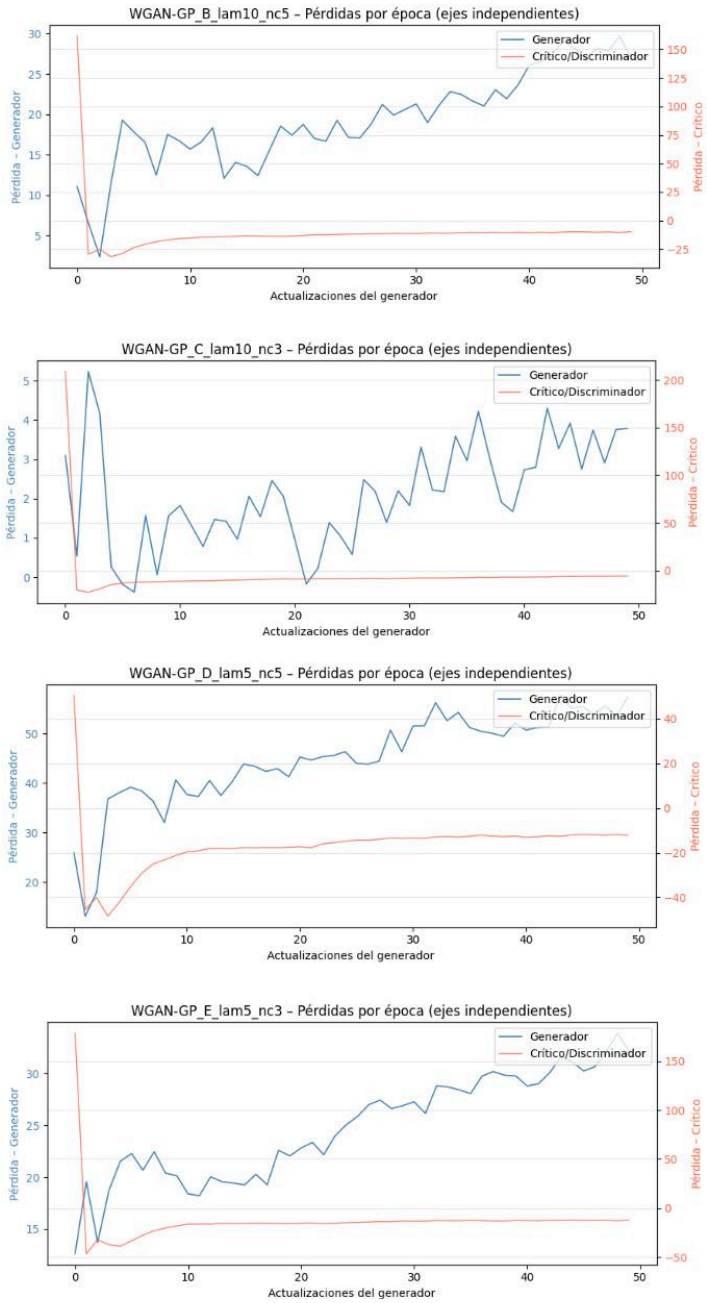


Figura 8. WGAN-GP con distintas configuraciones de hiperparámetros: B, C, D y E.

B. *Evaluación cuantitativa*

La métrica principal utilizada es el Fréchet Inception Distance (FID), que permite comparar las distribuciones de imágenes reales y sintéticas en un espacio de características. Un menor valor de FID indica una mayor similitud estadística entre ambas distribuciones. Adicionalmente se reportan la varianza promedio de píxeles y la distancia promedio entre imágenes como métricas de diversidad generativa.

La varianza promedio de píxeles mide la dispersión entre imágenes generadas: valores cercanos a cero indican mode collapse (imágenes casi idénticas entre sí), mientras que valores más altos indican mayor diversidad. La distancia promedio entre imágenes complementa esta métrica midiendo la diferencia en el espacio de píxeles entre pares de imágenes generadas. El baseline DCGAN sirve como referencia de comparación.

La diversidad de las imágenes generadas se evalúa de forma indirecta mediante el FID, el cual penaliza tanto la baja calidad visual como el colapso de modos. Cabe destacar que FID presenta limitaciones en dominios médicos, dado que InceptionV3 está entrenado en imágenes del mundo cotidiano (ImageNet) y no específicamente en imágenes oftalmológicas.

VI. Resultados y discusión

Si bien las configuraciones WGAN-GP B y C presentan parámetros muy similares, la configuración C alcanza mejor rendimiento con un menor número de actualizaciones del crítico (n_critic=3), lo que sugiere un equilibrio adversarial más adecuado entre generador y crítico. Aunque la configuración C no presenta los valores más altos de varianza ni de distancia entre imágenes, obtiene el menor FID, lo que indica que la diversidad generada se mantiene dentro de un dominio anatómico plausible.

Modelo	FID	Var. píxeles	Dist.
DCGAN (baseline)	47.40	0.013054	31.54
WGAN-clip (lr=5e-5, clip=0.01)	353.6	0.009537	30.05
WGAN-GP B (lambda_gp=10, nc=5)	90.46	0.010664	28.82
WGAN-GP C (lambda_gp=10, nc=3)	87.06	0.009515	27.66
WGAN-GP D (lambda_gp=5, nc=5)	90.71	0.012721	31.92
WGAN-GP E (lambda_gp=5, nc=3)	94.08	0.012156	31.31

Tabla 4. Resultados comparativos de todos los modelos (50 épocas)

DCGAN obtuvo el mejor FID (47.40), confirmando que en escenarios de datos limitados los modelos más simples pueden ser competitivos. WGAN-clip presentó colapso severo (FID=353.62) desde la época 5, ilustrando empíricamente la limitación del weight clipping.

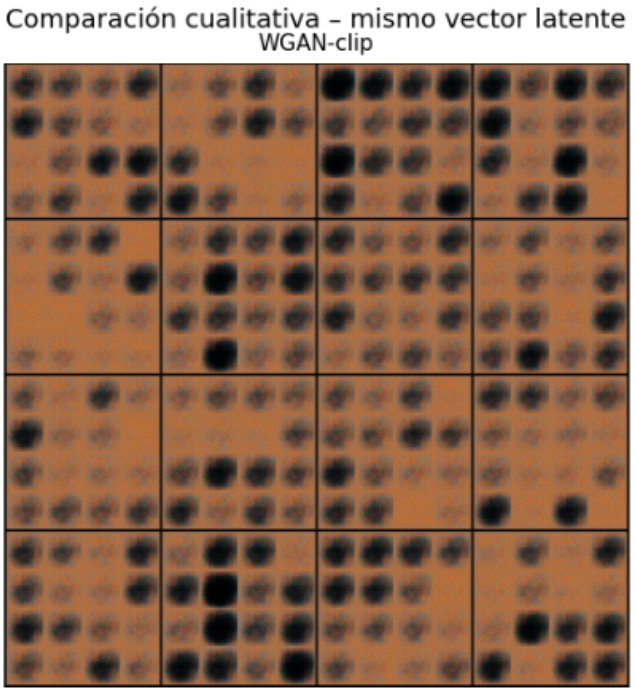
Entre las configuraciones de WGAN-GP, la config C (lambda_gp=10, n_critic=3) obtuvo el mejor FID (87.06), sugiriendo que un crítico menos frecuente genera mejor equilibrio adversarial con este dataset pequeño. La config D (lambda_gp=5, n_critic=5) mostró LossG escalando hasta valores de 57 al final del entrenamiento, indicando dominancia del crítico, aunque con la mayor diversidad entre configs

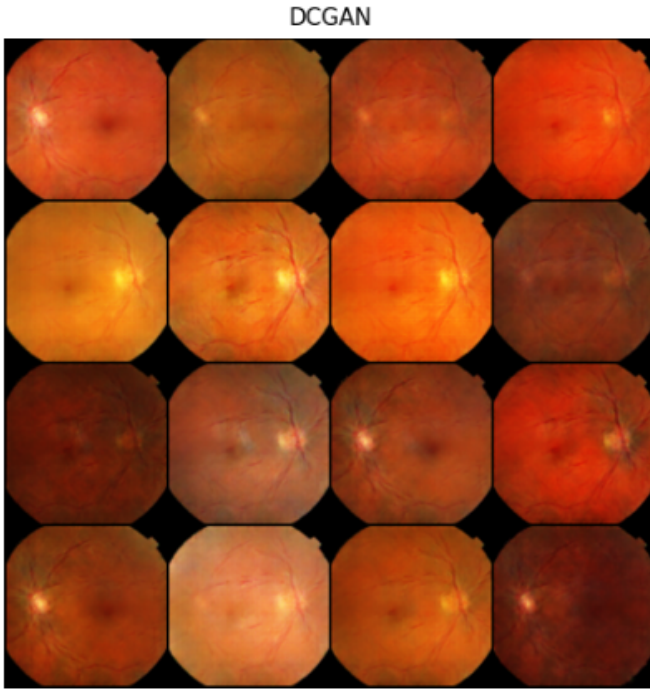
WGAN-GP (Dist. inter-img=31.92, superando incluso a DCGAN).

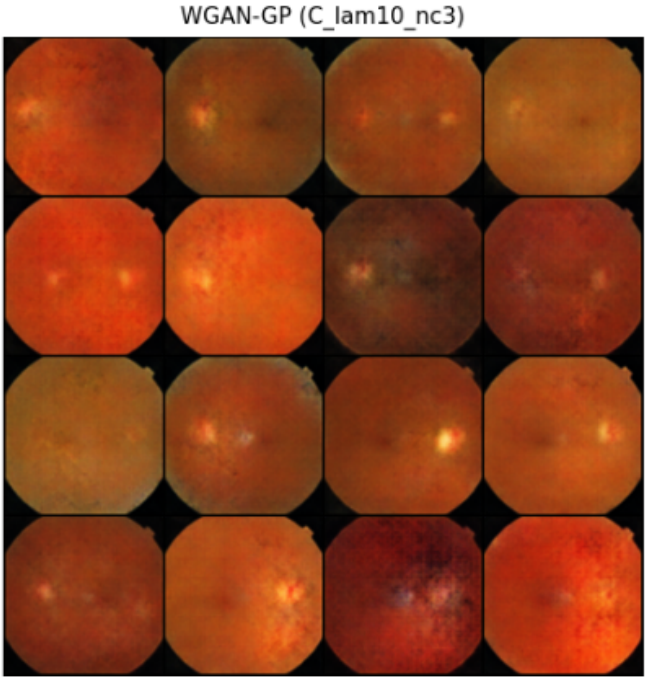
La corrección de BatchNorm por InstanceNorm redujo el FID de WGAN-GP de 138.14 (versión inicial) a 87.06 (config C), una mejora de 51.08 puntos, confirmando que el error de normalización era la causa principal del bajo desempeño original.

La comparación cualitativa revela diferencias consistentes entre las arquitecturas evaluadas. WGAN-clip produce patrones repetitivos y manchas no anatómicas que rompen la coherencia de las imágenes con una marcada pérdida de estructura retinal, a diferencia de DCGAN que genera retinas con morfología reconocible.

WGAN-GP (configuración C) proporciona imágenes que conservan anatomía retinal, presenta un disco óptico definido y muestra variaciones en textura y color. Dado que las imágenes se obtuvieron usando el mismo vector latente, estas diferencias se atribuyen al esquema de entrenamiento y a la función de pérdida.







VII. Conclusiones

En este trabajo se implementaron y compararon tres modelos generativos adversariales, DCGAN, WGAN-clip y WGAN-GP, para la síntesis de imágenes de fondo de ojo sobre el dataset Messidor-2, con el objetivo de contribuir a mitigar la escasez de datos en imágenes médicas oftalmológicas.

Los resultados evidencian dos patrones: lambda_gp=10 produce mejor FID en ambas configuraciones (B y C vs D y

E), indicando que una penalización más fuerte estabiliza mejor el entrenamiento.

Dado el tamaño reducido del dataset Messidor-2, n_critic=5 genera demasiado desequilibrio, debido a que el crítico aprende rápido las pocas imágenes reales y domina al generador. Con n_critic=3 da menos ventaja al crítico y se mantiene mejor el equilibrio adversarial. Esto se ve al tener n_critic=3 superando a n_critic=5 con el mismo lambda, sugiriendo que el dataset se beneficia de actualizaciones menos frecuentes del crítico para mantener el equilibrio adversarial.

Se pudo apreciar los efectos del uso incorrecto de BatchNorm en el crítico de WGAN-GP y la mejora al sustituirlo por InstanceNorm2d, reduciendo el FID en 51 puntos.

WGAN-clip confirmó su limitación conocida con colapso severo desde la época 5. WGAN-GP con la configuración correcta (lambda_gp=10, n_critic=3, InstanceNorm) se aproxima al baseline DCGAN pero sin superarlo bajo las condiciones experimentales consideradas. La exploración de hiperparámetros evidencia que lambda_gp=10 y n_critic=3 constituyen la combinación más adecuada para datasets médicos pequeños.

La superioridad teórica de WGAN-GP no se traduce automáticamente en mejor desempeño cuantitativo en escenarios de datos limitados, resaltando la importancia de la configuración experimental y la normalización correcta del crítico.

Como resultado final se generó un dataset sintético de 2.000 imágenes de fondo de ojo con la configuración C (lambda_gp=10, n_critic=3).

Como trabajo futuro se podría:

- Explorar GANs condicionales para generar imágenes por grado de severidad.
- Ajustar hiperparámetros de WGAN-GP.
- Evaluar métricas adicionales específicas del dominio médico.
- Incorporar validación clínica con especialistas.
- Implementar evaluación downstream mediante entrenamiento de un clasificador sobre el dataset sintético generado.

En resumen, el estudio confirma que los modelos generativos adversariales constituyen una alternativa viable para mitigar la escasez de datos en imágenes médicas, aunque su desempeño depende críticamente del diseño experimental y la naturaleza del conjunto de datos.

Referencias

[1] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial nets,” in Advances in Neural Information Processing Systems, vol. 27, 2014.

[2] A. Radford, L. Metz, and S. Chintala, “Unsupervised representation learning with deep convolutional generative adversarial networks,” in Proc. International Conference on Learning Representations (ICLR), 2016.

[3] M. Arjovsky, S. Chintala, and L. Bottou, “Wasserstein generative adversarial networks,” in Proc. International Conference on Machine Learning (ICML), 2017.

[4] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville, “Improved training of Wasserstein GANs,” in Advances in Neural Information Processing Systems, vol. 30, 2017.

[5] M. Arjovsky and L. Bottou, “Towards principled methods for training generative adversarial networks,” arXiv:1701.04862, 2017.

[6] T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, and X. Chen, “Improved techniques for training GANs,” in Advances in Neural Information Processing Systems, vol. 29, 2016.

[7] E. Decenciere et al., “Feedback on a publicly distributed image database: The Messidor database,” Image Analysis & Stereology, vol. 33, no. 3, pp. 231–234, 2014

[8] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, “GANs trained by a two time-scale update rule converge to a local Nash equilibrium,” in Advances in Neural Information Processing Systems, vol. 30, 2017.

En el desarrollo de este trabajo se utilizaron asistentes de IA como ChatGPT, Claude y Gemini como herramientas de apoyo en tareas de depuración, ampliación de búsqueda bibliográfica, testing, refactorización de código y como corrector de redacción. Todas las decisiones metodológicas, experimentos, implementaciones y conclusiones son responsabilidad del autor.

Generador:

```
Generador(
  (net): Sequential(
    (0): ConvTranspose2d(100, 512, kernel_size=(4, 4), stride=(1, 1), bias=False)
    (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU(inplace=True)
    (3): ConvTranspose2d(512, 256, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (4): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (5): ReLU(inplace=True)
    (6): ConvTranspose2d(256, 128, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (7): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (8): ReLU(inplace=True)
    (9): ConvTranspose2d(128, 64, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (10): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (11): ReLU(inplace=True)
    (12): ConvTranspose2d(64, 32, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (13): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (14): ReLU(inplace=True)
    (15): ConvTranspose2d(32, 3, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (16): Tanh()
  )
)
```

Discriminador DCGAN:

```
D_DCGAN(
  (net): Sequential(
    (0): Conv2d(3, 64, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (1): LeakyReLU(negative_slope=0.2, inplace=True)
    (2): Conv2d(64, 128, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (3): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (4): LeakyReLU(negative_slope=0.2, inplace=True)
    (5): Conv2d(128, 256, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (6): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (7): LeakyReLU(negative_slope=0.2, inplace=True)
    (8): Conv2d(256, 512, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (9): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (10): LeakyReLU(negative_slope=0.2, inplace=True)
    (11): Conv2d(512, 1024, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (12): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (13): LeakyReLU(negative_slope=0.2, inplace=True)
```

```
    (14): Conv2d(1024, 1, kernel_size=(4, 4), stride=(1, 1), bias=False)
    (15): Sigmoid()
  )
)
```

Crítico WGAN (InstanceNorm):

```
Critic_WGAN(
  (net): Sequential(
    (0): Conv2d(3, 64, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (1): LeakyReLU(negative_slope=0.2, inplace=True)
    (2): Conv2d(64, 128, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (3): InstanceNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=False)
    (4): LeakyReLU(negative_slope=0.2, inplace=True)
    (5): Conv2d(128, 256, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (6): InstanceNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=False)
    (7): LeakyReLU(negative_slope=0.2, inplace=True)
    (8): Conv2d(256, 512, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (9): InstanceNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=False)
    (10): LeakyReLU(negative_slope=0.2, inplace=True)
    (11): Conv2d(512, 1024, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (12): InstanceNorm2d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=False)
    (13): LeakyReLU(negative_slope=0.2, inplace=True)
    (14): Conv2d(1024, 1, kernel_size=(4, 4), stride=(1, 1), bias=False)
  )
)
```